Reviewer Pack — Minimal Rank of Quadratic Forms over Noncommutative Algebras…

Entry 6fe510ed8267 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 16:28:50 UTC

Minimal Rank of Quadratic Forms over Noncommutative Algebras vs BP_ReadTwice Circuit Size

Entry ID: `6fe510ed8267` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 16:28:50 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

Statement:

> ['For every read-twice branching program P with size n, the minimal rank of the quadratic form defined over the algebraic structure associated with P is O(log n). Specifically, for all k-CNF instances, if P is a read-twice BP solving k-CNF in polynomial time, then there exists a noncommutative algebraic structure such that the minimal rank of its corresponding quadratic form is O(log n^k).', 'Equivalently, for every instance I with n variables and m clauses, the minimal rank of the quadratic form derived from the algebraic structure representing the read-twice BP solving I is at least log(n) * log(m).']

Rationale (proposer's reasoning):

> ["Noncommutative algebras offer a rich framework to study algebraic structures associated with Boolean functions. The minimal rank of a quadratic form can serve as an invariant that captures the complexity of a read-twice BP's behavior, potentially revealing hidden complexities in polynomial-time solvable problems.", 'This connection between noncommutative algebra and computational complexity could expose new structures and methods to understand the inherent difficulty of computational problems.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bffba1b6f9a01dd1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a read-twice BP solving k-CNF, the minimal rank of its quadratic form over the associated noncommutative algebra is O(log n^k).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0 for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def min_rank(A):
        m, n = len(A), len(A[0])
        rank = 0
        for i in range(min(m, n)):
```

```
            if A[i][i] != 0:
                rank += 1
        return rank

    n = random.randint(5, 40)
    m = random.randint(n, n * 2)

    # Generate a random k-CNF instance
    variables = [f'x{i}' for i in range(n)]
    clauses = []
    for _ in range(m):
        clause = random.sample(variables + [f'¬{v}' for v in variables], 2)
        clauses.append(clause)

    # Construct a read-twice BP (simplified representation)
    bp_size = len(clauses) * n

    # Derive the algebraic structure (simulated as a matrix)
    A = [[0 for _ in range(bp_size)] for _ in range(bp_size)]
    for i, clause in enumerate(clauses):
        for j, var in enumerate(variables):
            if var in clause:
                A[i*n + j][i*n + j] += 1
            if f'¬{var}' in clause:
                A[i*n + j][(i+1)*n + (j+1) % n] += 1

    # Compute the minimal rank of the quadratic form
    rank = min_rank(A)

    return {
        "metric_name": "minimal_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank <= math.log(n**len(clauses)),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 50, 2))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_rank = sum(r['metric_value'] for r in results) / len(results)
    std_rank = math.sqrt(sum((r['metric_value'] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    else:
```

```
        first_failing_seed = next(i for i, r in enumerate(results) if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={seeds[first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {"seed": 11, "metric_name": "minimal_rank", "metric_value": 71, "instances_tested": 1, "conjec

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e8db499.py", line 91, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e8db499.py", line 72, in run_trial
    A[i*n + j][(i+1)*n + (j+1) % n] += 1
    ~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError before producing valid data; pre-registered support conditions cannot be evaluated. | next: Fix the IndexError in test_7e8db499.py by validating array bounds during matrix construction

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15412
2	preregistration	ollama_remote	glm4:latest	0	0	9373
3	novelty	ollama_remote	glm4:latest	0	0	8619
4	novelty	ollama_remote	glm4:latest	0	0	8604
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13518
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8707
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7675
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11265
9	judge	ollama_local	qwen3:8b	0	0	396123

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 479296 ms total latency. Provider mix: {'ollama_remote': 8, 'ollama_local': 1}

(full prompt+response transcripts available in `research/audit/6fe510ed8267.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6fe510ed8267.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6fe510ed8267.tar.gz` (if generated)